

Olist Brazilian E-Commerce Dataset

Dataset Overview & Table Relationships — Olist E-Commerce SQL Analytics Project

1. What Is the Olist Dataset?

The Olist Brazilian E-Commerce Public Dataset is a real, anonymized collection of order data from Olist, a marketplace that connects small and medium-sized Brazilian merchants to major online sales channels. Rather than a single flat file, it is a **relational dataset**: information about a single order is deliberately spread across nine linked tables, mirroring the way a production e-commerce database is actually structured.

This makes it well suited to SQL analysis. Instead of reading pre-aggregated numbers from one sheet, the tables have to be joined and reasoned about together, which reflects the day-to-day work of a data analyst querying a real transactional system.

For this project, the nine source tables were loaded into a MySQL database (**olist_ecommerce**) and used to answer business questions spanning the full order lifecycle — from the moment a customer places an order to payment, fulfillment, delivery, and the review left afterward.

2. Dataset Scale

The dataset covers roughly 100,000 orders placed between 2016 and 2018. The table below summarizes the approximate size and content of each table.

Table	Approx. Records	Main Information
customers	99,441	Customer and location information
orders	99,441	Order status, purchase and delivery dates
order_items	112,650	Products, sellers, prices and freight
order_payments	103,886	Payment methods, installments and values
order_reviews	~100,000	Ratings, comments and review timestamps
products	32,951	Categories and physical product attributes
sellers	3,095	Seller location information
geolocation	1,000,163	ZIP-code-level geographic coordinates
category_translation	71	Portuguese-to-English category names

Record counts are approximate and may vary slightly after data cleaning, or depending on how duplicate or geolocation records are handled.

3. Main Tables and What They Represent

customers	One row per customer order record, holding a customer ID, a persistent unique customer ID, and city/state location. The two ID fields exist because Olist assigns a new <code>customer_id</code> per order, while <code>customer_unique_id</code> identifies the same real person across multiple orders — the key to detecting repeat customers.
orders	The central transaction table. Every order has a status (e.g. delivered, shipped, canceled) and a chain of timestamps — purchase, approval, shipping, delivery — that together allow delivery-time and delay analysis.
order_items	Line-item detail connecting each order to the specific products and sellers involved, including unit price and freight (shipping) value. An order can contain multiple items from multiple sellers.
order_payments	Payment detail per order: payment type (credit card, boleto, voucher, debit card), number of installments, and payment value. An order can have more than one payment record.
order_reviews	Customer feedback after delivery, including a 1–5 review score, optional comment text, and review timestamps — the main signal for customer satisfaction.
products	Catalog-level attributes: category name, dimensions, weight, and photo/description length. Category names are stored in Portuguese in this table.
sellers	Seller identifiers and their city/state location, used to analyze seller performance geographically.
geolocation	ZIP-code-prefix-level latitude/longitude, city and state, usable to map customers and sellers geographically.
category_translation	A small lookup table translating Portuguese product category names into English equivalents.

4. How the Tables Connect

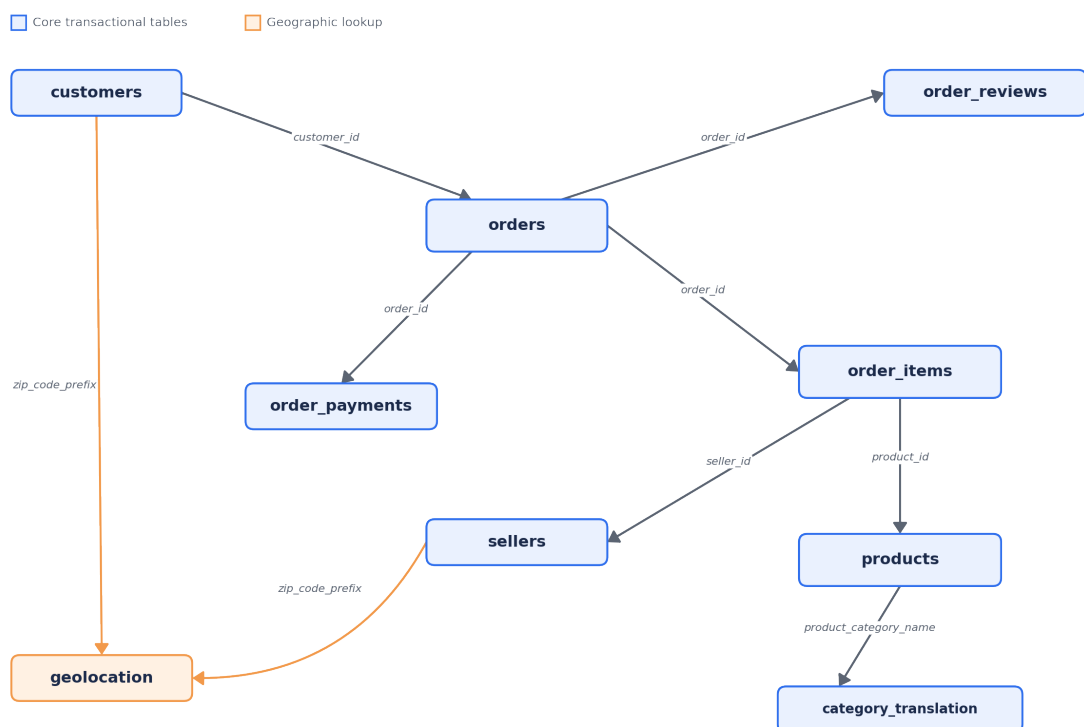
The **orders** table sits at the center of the schema and acts as the hub that everything else joins through. Every other transactional table links back to an order, and orders link back to a customer — so a single order can be traced end-to-end: who bought it, what was bought, from whom, how it was paid for, and how the customer rated the experience.

Key relationships

- **customers** → **orders** — joined on `customer_id`. Each order belongs to exactly one customer.
- **orders** → **order_items** — joined on `order_id`. One order can contain several line items (one-to-many).
- **order_items** → **products** — joined on `product_id`, identifying what was purchased.
- **order_items** → **sellers** — joined on `seller_id`, identifying who fulfilled that item.

- **orders** → **order_payments** — joined on `order_id` (one-to-many: an order may be split across multiple payments).
- **orders** → **order_reviews** — joined on `order_id`, linking delivery outcomes to customer satisfaction.
- **products** → **category_translation** — joined on `product_category_name`, converting Portuguese category names to English.
- **customers / sellers** → **geolocation** — joined on ZIP-code prefix, enabling state- and city-level geographic analysis.

Relationship diagram



orders is the hub of the schema: customer, payment and review data join directly to it, while product and seller detail join through order_items. geolocation and category_translation are lookup tables — geolocation is joined by ZIP-code prefix from both customers and sellers, and category_translation is joined to products by category name.

5. Business Questions This Dataset Can Answer

- How much revenue is generated over time?
- Which product categories and products generate the most sales?
- Which customers are the highest-value customers?
- What percentage of customers are repeat customers?
- Which sellers generate the highest revenue?
- Which payment methods are most commonly used?
- Which states generate the most sales?
- How long does delivery take, and how often are orders delivered late?
- Does delivery performance relate to customer review scores?
- Which categories or sellers have stronger or weaker customer satisfaction?

6. Why This Dataset Is Useful for an SQL Project

Because it is relational and spans multiple business dimensions, this dataset supports a realistic range of SQL techniques — joins across up to eight tables at once, GROUP BY and aggregate functions, subqueries and CTEs, CASE expressions, window functions and ranking, date-based delivery analysis, and indexing for query optimization. It is close enough to a real production schema that the resulting queries reflect the kind of work an analyst does against a live transactional database, rather than a single denormalized spreadsheet.

7. Project Presentation Opening

Suggested explanation:

“For this project, I worked with the Olist Brazilian E-Commerce Public Dataset. It is a real-world e-commerce dataset containing nearly 100,000 orders and multiple related tables covering customers, products, sellers, payments, reviews and delivery. Instead of analyzing a single CSV, I designed a relational MySQL database and connected these tables to investigate the complete e-commerce journey — from customer and order information to payment, product, seller, delivery and customer satisfaction. The goal of the project is to convert raw transactional data into meaningful business insights using SQL.”

8. Project Technology

Database	MySQL 8
Analysis	SQL
Data Cleaning	Python / Pandas, where required
Dataset	Olist Brazilian E-Commerce Public Dataset
